TD2: Base de données orientées document

November 9, 2026



1 La BDOG mongoDB

MongoDB est une **base de donnée orientée document**. Elle fonctionne avec deux types d'objet principaux les **collections** et **les documents**.

- Chaque collection est une nouvelle base de documents.
- Les documents sont au format **JSON**.
- **Les document ne sont soumis à aucun modèle préalable**, ils doivent uniquement posséder une clé **_id** !

Question 1 Quels sont les intérêts de ne pas avoir de modèle de données.
Question 2 Quels sont les inconvénients.

1.1 Installer MongoDB

Le plus simple est de lancer **un shell interactif** sur une instance de **MngoDB exécutée dans un container Docker** !

Lancer l'image mongodb dans un conteneur :

```
docker run --name mongodb -p 27017:27017 mongodb/mongodb-community-server:latest
```

```
docker run --name mongodb -p 27017:27017
-e MONGODB_INITDB_ROOT_USERNAME=admin
-e MONGODB_INITDB_ROOT_PASSWORD=secret
--privileged
-d
mongodb/mongodb-community-server:latest
```

Utiliser le shell interactif mogosh du conteneur:

```
docker exec -it mongodb mongosh
```

2 Interface CRUD

MongoDB est munie d'une interface **CRUD** il existe donc des fonctions pour manipuler chaque collection comme il existe des fonctions pour manipuler chaque table d'une base de données relationnelle.

Vous trouverez une documentation sur cette interface à l'adresse suivante:
https://www.mongodb.com/docs/manual/crud/

2.1 Insertion

Insérez un nouveau document :

```
// Creation d'une nouvelle collection
use td2

// Exemple d'article de sport a inserer dans la collection
db.td2.insertOne({
    "nom": "T-shirt Sport Homme",
    "categorie": "vetements",
    "prix": 25.99,
    "marque": "Nike",
    "taille": ["S", "M", "L", "XL"],
    "disponibilite": true
})

db.td2.find({})
```

Question 1: Expliquez le résultat de la commande précédente.

Question 2: Insérer les articles contenus dans le fichier **articles.json** avec la fonction **insertMany**.

```
db.td1.insertMany([
        { ... },
        ...
]}
```

2.2 Sélection

Les opérations de lecture utilisent la commande **find**, cette commande prend en paramètre un dictionnaire qui contient les clés des documents utilisés comme critère et comme valeur la valeur recherchée **(ou un prédicat)** .

Par défaut tout document qui "match" est retourné !

Question 1 Trouvez tous les articles de la marque Nike.

Question 2 Trouvez toutes les chaussure de la marque Nike.

Question 3 Trouvez tous les articles dont le prix est supérieur à 20.0 euros et inférieur à 30 euros.

Question 4 Trouvez les articles qui correspondent de marque Adidas entre 20 et 30 euros.

Question 5 Trouvez les articles disponibles en S ou XS

Le deuxième paramètre de la fonction find est un dictionnaire qui indique quels champs du document doit être retourné. Il doit contenir soit tout les champs inclus ou exclus (à l'exception de id).

Question 6 Retourner tout les champs à l'exception de **_id**

Question 7 Retourner uniquement la **marque** le **nom** et le **prix**.

2.3 Mise à jour

De la même manière que pour l'insertion on peut modifier un document en particulier ou une liste de document qui répondent à des critères.

Les fonctions utilisées sont les suivantes:

```
db.collection.updateOne()

db.collection.updateMany()
```

Set permet d'ajouter un champs ou de le modifier si il existe

Question 1 Modifiez le prix d'un article.

Question 2 Ajoutez à un article un tableau de couleurs

Question 3 Ajoutez un champs qt initialisé à 100 pour chaque article.

Question 4 Ajoutez 20% au prix de chaque article.

2.4 Suppression

La suppression d'un document utilise les fonctions :

```
db.collection.deleteOne()

db.collection.deleteMany()
```

Ces fonctions utilisent un filtre en premier argument, dans le cas de **deleteOne** c'est le premier document trouvé qui est supprimé.

Question 1 Supprimez un document spécifique.

Question 2 Supprimez toutes les chaussures.